## Table of Contents

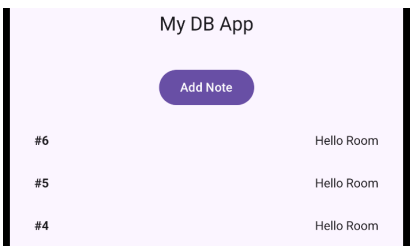
**SQLITE with the Room Library** ..... **1**
Obtaining the Necessary Dependencies ..... 1
Creating a Data Entity ..... 2
Defining the DAO ..... 2
Inserting and Selecting All Items ..... 2
Defining the Database ..... 2
Deleting an Item ..... 8
Querying an Item by ID ..... 10
Updating an Item ..... 13

## SQLITE with the Room Library

The Kotlin team recommends using Room database which is a library that provides an efficient and simplified interface for working with Sqlite.

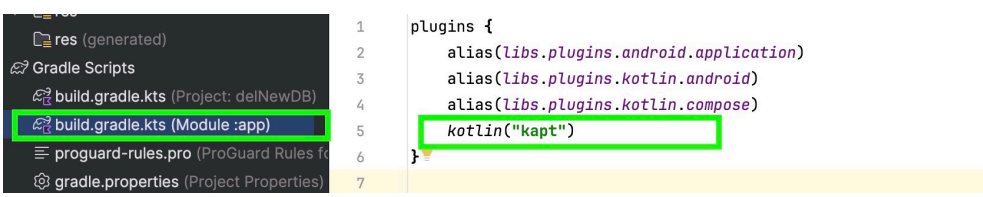
### *Example 1: Working with Room Database*

This example sets up the database, how to add rows to a table and how to view rows of data from it.




## Obtaining the Necessary Dependencies

Starting with the dependencies, kapt must be added to the plugins section of the build.gradle file at the app level. kapt processes annotations which define code structures. kapt will allow us to use several database annotations including @Entity, @Dao and @Query.




```
kotlin("kapt")
```

Next, in the dependencies section, import the required room dependencies and then press **Sync Now**:

```
val room_version = "2.8.3"

implementation("androidx.room:room-runtime:$room_version")
implementation("androidx.room:room-ktx:$room_version")
annotationProcessor("androidx.room:room-compiler:$room_version")
kapt("androidx.room:room-compiler:$room_version")
implementation("androidx.lifecycle:lifecycle-runtime-ktx:$room_version")
```

### Creating a Data Entity

An **data entity class** in Kotlin represents a database table template. Entity classes automate functions for storing and managing data through a code interface.

**Note.kt**

```
@Entity
data class Note(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val text: String
)
```

### Defining the DAO

In a file called NoteDao, the data access object is defined. Dao objects contain database queries. The annotations, **@Insert**, **@Query**, **@Update** and **@Delete** help to enable create, read, update and delete (CRUD) database operations.

### Inserting and Selecting All Items

Here is a starter DAO which performs an insert and select all for rows of the Note table.

**NoteDao.kt**

```
@Dao
interface NoteDao {
    @Insert
    suspend fun insert(note: Note)

    @Query("SELECT * FROM Note ORDER BY id DESC")
    suspend fun getNotes(): List<Note>
}
```

### Defining the Database

In a file called Database, add this code which will build the database if it does not exist. Code is also included to link the database with the NoteDao.

**Database.kt**

```
// Build database if it does not exist.
object DatabaseProvider {
    fun provideDB(context: Context): AppDB {
        return Room.databaseBuilder(
            context.applicationContext,
            AppDB::class.java,
            "notes.db"
        ).build()
    }
}

// Link database with the DAO.
@Database(entities = [Note::class], version = 1)
abstract class AppDB : RoomDatabase() {
    abstract fun noteDao(): NoteDao
}
```

To ensure the data displays properly in a scrollable list, in MainActivity.kt add this import statement.

**MainActivity.kt**

```
import androidx.compose.foundation.lazy.items
```

Next, replace the rest of the code in MainActivity.kt with this version which combines the database query launch code with the composable UI. The database query code is highlighted in yellow.

**MainActivity.kt**

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent { NotesApp() }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun NotesApp() {
    val context = LocalContext.current
    val db = remember { DatabaseProvider.provideDB(context) }

    var notes by remember { mutableStateOf<List<Note>>(emptyList()) }
    val scope = rememberCoroutineScope()

    suspend fun reload() {
        val data = db.noteDao().getNotes()
        withContext(Dispatchers.Main) { notes = data }
    }

    // Run reload() at startup. LaunchedEffect is for start up.
    LaunchedEffect(Unit) {
        reload()
    }

    Scaffold(
        topBar = {
            CenterAlignedTopAppBar(
                title = { Text("My DB App") }
            )
        }
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
                .padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
                Button(onClick = {
                    // Scope ties coroutine to composable. Launch starts coroutine.
                    scope.launch(Dispatchers.IO) {
                        db.noteDao().insert(Note(text = "Hello Room"))
                        reload()
                    }
                }) { Text("Add Note") }
```

*Exercise 1 (1 mark)*

What annotation is used to determine how a class is defined as a template for a database table?

@Entity

*Exercise 2 (1 mark)*

What dependency is required to enable annotations such as @Insert or @Query?

Kapt

*Exercise 3 (1 mark)*

How are columns designated as primary keys in an entity?

Using @PrimaryKey annotation on the property. Adding autoGenerate = true makes Room automatically assign a unique number to each row.

*Exercise 4 (1 mark)*

What is the role of the @Dao annotation and what does DAO stand for?

It marks an interface as a Data Access Object. It stands for Data Access Object

*Exercise 5 (1 mark)*

If you were to add a dayOfWeek column as a string to the Note entity, show what the Note entity class would look like.

```
@Entity
data class Note(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val text: String,
    val dayOfWeek: String
)
```

*Exercise 6 (1 mark)*

How is the purpose of LaunchedEffect(Unit) different from a suspend function?

LaunchedEffect(Unit) runs once when the screen starts which is for startup tasks like loading data. A suspend fun is just a function that can run in the background. but it doesn't automatically run on startup, we have to call it ourselves.

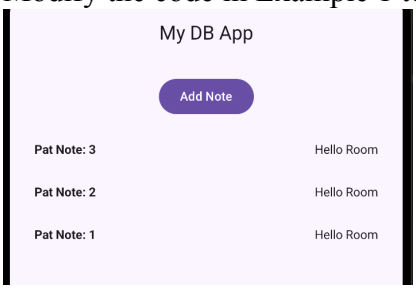
*Exercise 7 (1 mark)*

What is the role of the scope function?

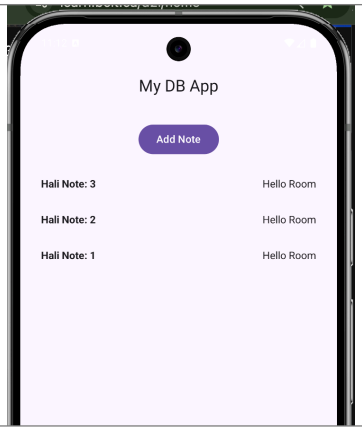
| |
|---|
| runs background tasks and auto-cancels them when the screen closes. |

*Exercise 8 (2 marks)*

Modify the code in Example 1 to show your name along with the id.




Show a screenshot of the output after making this change.

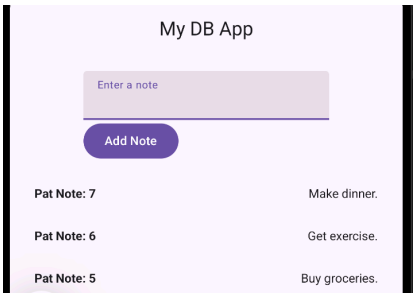



Show the code used to make this change.

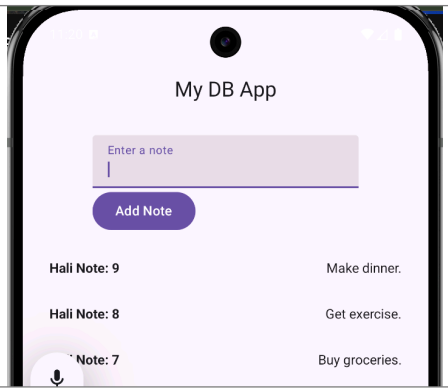
```
Text(
    "Hali Note: ${n.id}",
    fontWeight = FontWeight.Bold
)
Text(n.text)
```

*Exercise 9 (4 marks)*

Add a TextField composable to the interface from Example 1 so it can receive and submit custom content to the database.




Show a screenshot of the interface after making this change and after adding some custom content of your own.



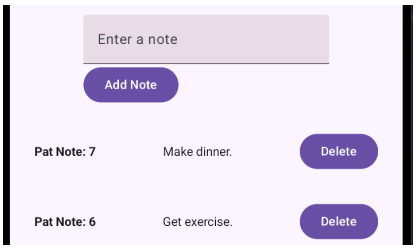

Show the code used to make this change including the state variable declaration.

```
- var mynote by remember { mutableStateOf("") }
- Column {
    Row(horizontalArrangement =
Arrangement.spacedBy(12.dp)) {
        TextField(
            value = mynote,
            onValueChange = { mynote = it },
            label = { Text("Enter a note") }
        )
    }
    Row(horizontalArrangement =
Arrangement.spacedBy(12.dp)) {
        Button(onClick = {
            scope.launch(Dispatchers.IO) {
                db.noteDao().insert(Note(text = mynote))
                reload()
                mynote = ""
            }
        }) { Text("Add Note") }
    }
}
```

## Deleting an Item

*Example 2: Adding a Delete Function*

The example continues from Example 1 to add a delete button.




To **NoteDao.kt**, add a delete function:

```
@Delete
suspend fun delete(note: Note)
```

Within the Row() object of the LazyColumn inside **MainActivity.kt**, add this code to delete a database row when the delete button is clicked.

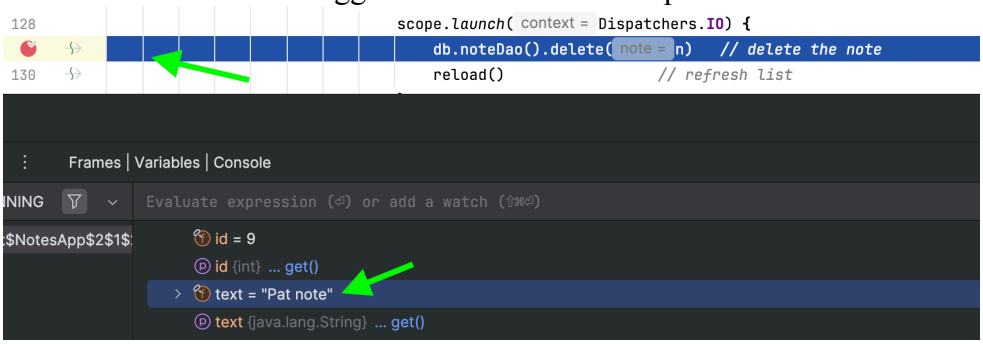
```
Button(onClick = {
    scope.launch(Dispatchers.IO) {
        db.noteDao().delete(n)   // delete the note
        reload()                 // refresh list
    }
}) {
    Text("Delete")
}
```

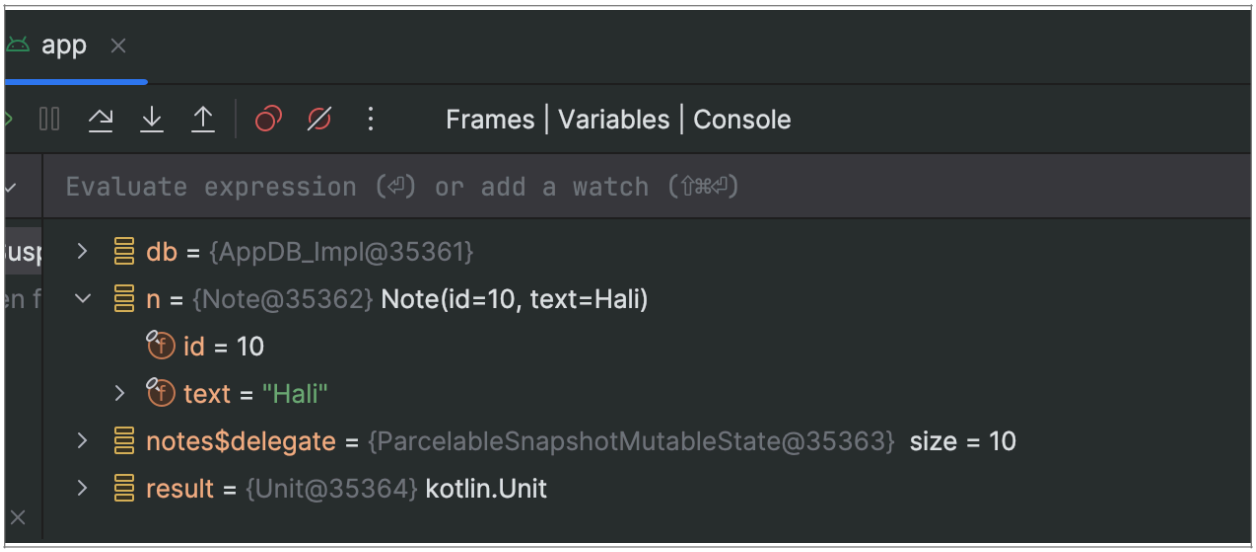
The row can be centered with this addition:

```
verticalAlignment = Alignment.CenterVertically
```

*Exercise 10 (2 marks)*

Create a note with your name in it. Then set a break point in the area where it is deleted. Show a screenshot while the debugger is halted at the breakpoint where the item is deleted.

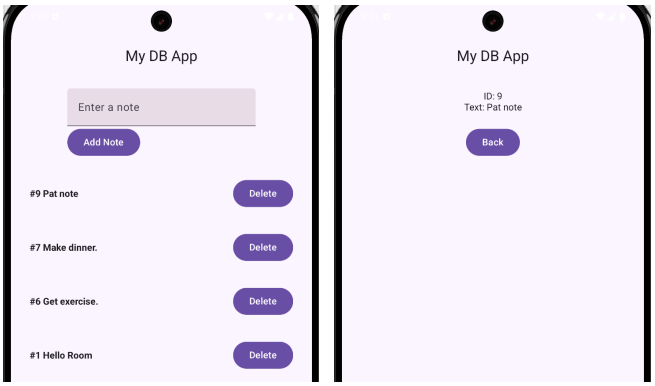





## Querying an Item by ID

*Example 3: Querying an Item by ID*

This example demonstrates how to enable the querying of an item by ID for display in a detail view.




To enable the search by id query, add this code to NoteDao.

**NoteDao.kt**

```
@Query("SELECT * FROM Note WHERE id = :id LIMIT 1")
suspend fun getNoteById(id: Int): Note?
```

Then in a separate file, add this code to display a detail view if the row for the Note object is selected. While it is possible to use the Navigator class to display the detail screen, this example simply swaps UI to show details for a selected object when a row is clicked.

**NoteDetail.kt**

```
@Composable
fun NoteDetailScreen(
    id: Int,
    db: AppDB,
    onBack: () -> Unit
) {
    var note by remember { mutableStateOf<Note?>(null) }

    // Load note from DB when screen opens
    LaunchedEffect(id) {
        val n = withContext(Dispatchers.IO) { db.noteDao().getNoteById(id) }
        note = n
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(20.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Detail Screen", fontWeight = FontWeight.Bold)
        Spacer(Modifier.height(100.dp))

        if (note == null) {
            Text("Loading...")
        } else {
            Text("ID: ${note!!.id}")
            Text("Text: ${note!!.text}")
        }

        Spacer(Modifier.height(20.dp))
        Button(onClick = onBack) { Text("Back") }
    }
}
```

At the top of the NotesApp() function in **MainActivity.kt**, add this code:

```
var selectedNoteId by remember { mutableStateOf<Int?>(null) }
```

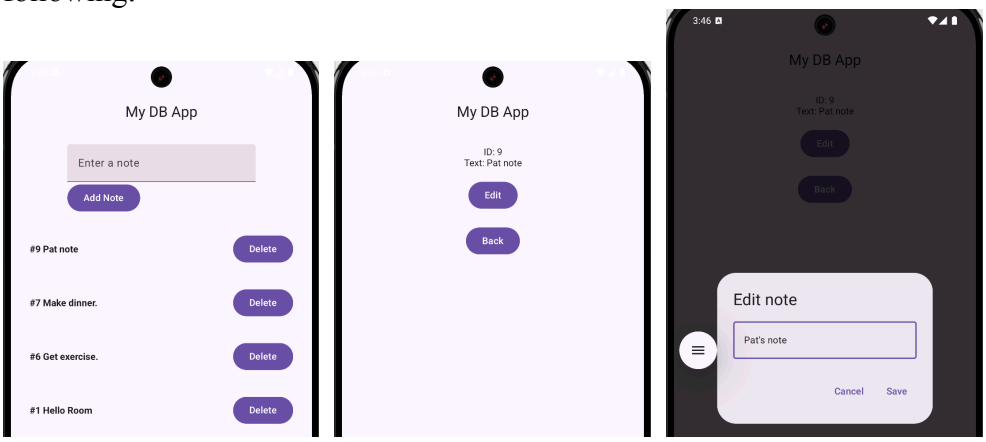
Then in **MainActivity.kt** replace the Column with code that performs a conditional toggle between the NoteDetailScreen and a list of notes. The new code that enables the toggle is highlighted in yellow.

```
if (selectedNoteId != null) {
    NoteDetailScreen(
        id     = selectedNoteId!!,
        db     = db,
        onBack = { selectedNoteId = null }
    )
}
else {

    Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(padding)
        .padding(16.dp),
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Column() {
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            TextField(
                value = mynote,
                onValueChange = { mynote = it },
                label = { Text("Enter a note") }
            )
        }
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            Button(onClick = {
                // Scope ties coroutine to composable. Launch starts coroutine.
                scope.launch(Dispatchers.IO) {
                    db.noteDao().insert(Note(text = mynote))
                    reload()
                    mynote = ""
                }
            }) { Text("Add Note") }
        }
    }

    Spacer(Modifier.height(16.dp))

    if (notes.isEmpty()) {
        Text("(no notes yet)")
    }
    else {
        LazyColumn(
            modifier = Modifier.fillMaxWidth(),
            verticalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                items(notes) { n ->
```

## Updating an Item

*Example 4: Updating an Item*

This example completes the database CRUD app by enabling an update that looks similar to the following:




To start, an update operation needs to be added to the data access object.

**NoteDao.kt**

```
@Update
suspend fun update(note: Note)
```

Next, add this code to define a view where the Note object can be updated.

**NoteEdit.kt**

```
@Composable
fun EditNoteDialog(
    initial: String,
    onConfirm: (String) -> Unit,
    onDismiss: () -> Unit
) {
    var text by remember { mutableStateOf(initial) }

    androidx.compose.material3.AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("Edit note") },
        text = {
            androidx.compose.material3.OutlinedTextField(
                value = text,
                onValueChange = { text = it },
                singleLine = true,
                modifier = Modifier.fillMaxWidth()
            )
        },
        confirmButton = {
            androidx.compose.material3.TextButton(onClick = { onConfirm(text) }) {
                Text("Save")
            }
        },
        dismissButton = {
            androidx.compose.material3.TextButton(onClick = onDismiss) {
                Text("Cancel")
            }
        }
    )
}
```

Lastly, replace the NoteDetailScreen.kt with this code. The highlighted code triggers the launch of the edit dialog and handles the user response.

```
@Composable
fun NoteDetailScreen(
    id: Int,
    db: AppDB,
    onBack: () -> Unit
) {
    var note by remember { mutableStateOf<Note?>(null) }
    var showEdit by remember { mutableStateOf(false) }             //   added
    val scope = rememberCoroutineScope()                           //   added

    // Load note from DB when screen opens
    LaunchedEffect(id) {
        val n = withContext(Dispatchers.IO) { db.noteDao().getNoteById(id) }
        note = n
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(20.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text("Detail Screen", fontWeight = FontWeight.Bold)
        Spacer(Modifier.height(100.dp))

        if (note == null) {
            Text("Loading...")
        } else {
            Text("ID: ${note!!.id}")
            Text("Text: ${note!!.text}")

            Spacer(Modifier.height(16.dp))

            // Edit button sets the showEdit variable to true.
            Button(onClick = { showEdit = true }) { Text("Edit") }
        }

        Spacer(Modifier.height(20.dp))
        Button(onClick = onBack) { Text("Back") }
    }

    // When showEdit==true, launch Edit dialog & handle response in 'onConfirm'.
    if (showEdit && note != null) {
        EditNoteDialog(
            initial = note!!.text,
            onConfirm = { newText ->
                scope.launch(Dispatchers.IO) {
                    db.noteDao().update(note!!.copy(text = newText))    // update
```

*Exercise 11 (1 mark)*

When you run the code in Example 4 you will notice that the main list is not updated right away after a row is updated. Where can this code be added in **MainActivity.kt** to refresh the list after an edit?

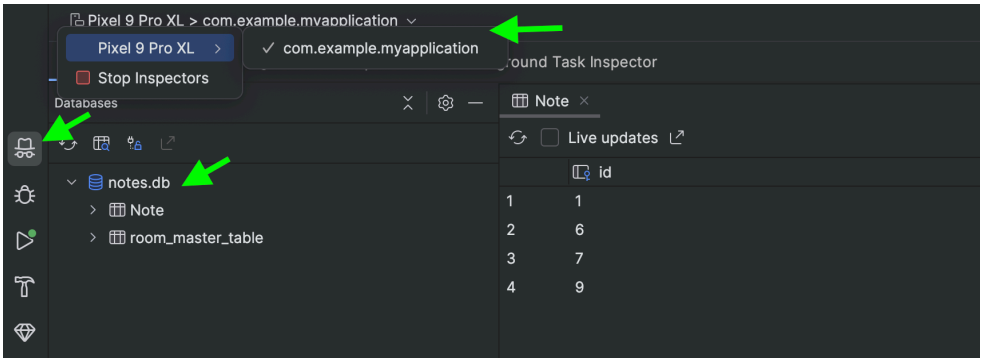
```
LaunchedEffect(Unit) {
    reload()
}
```

after the closing `)` of `NoteDetailScreen(…)`:

```
if (selectedNoteId != null) {
    NoteDetailScreen(
        id = selectedNoteId!!,
        db = db,
        onBack = { selectedNoteId = null }
    )
}
LaunchedEffect(Unit) {
    reload()
}
```

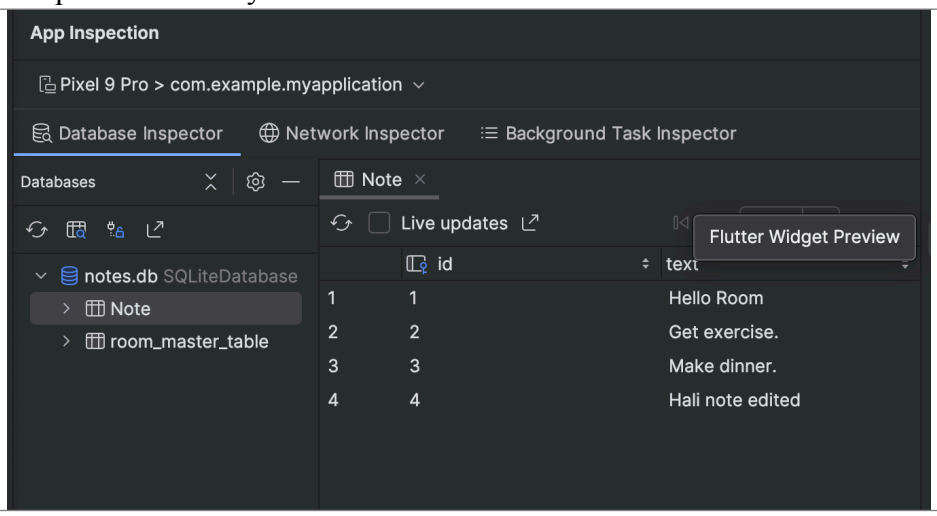
### *Example 5: Viewing the Database from Android Studio*

In Android Studio you can view the Sqlite database through **View | Tool Windows | App Inspection.** Then, once your emulator and app is selected, double clicking the table will show the data.




*Exercise 12 (2 marks)*

Add a field to the database with your first name in the task. Then take a screenshot of the table when in App Inspection. Show your screenshot here.

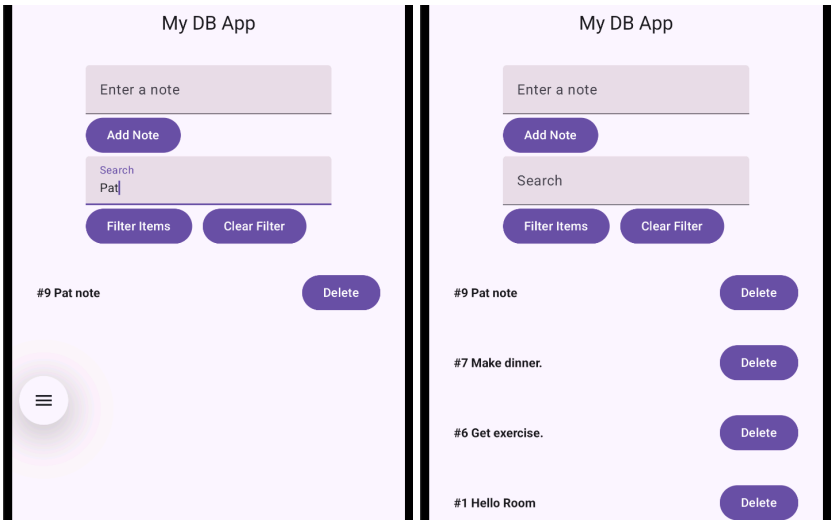



*Exercise 13 (8 marks)*

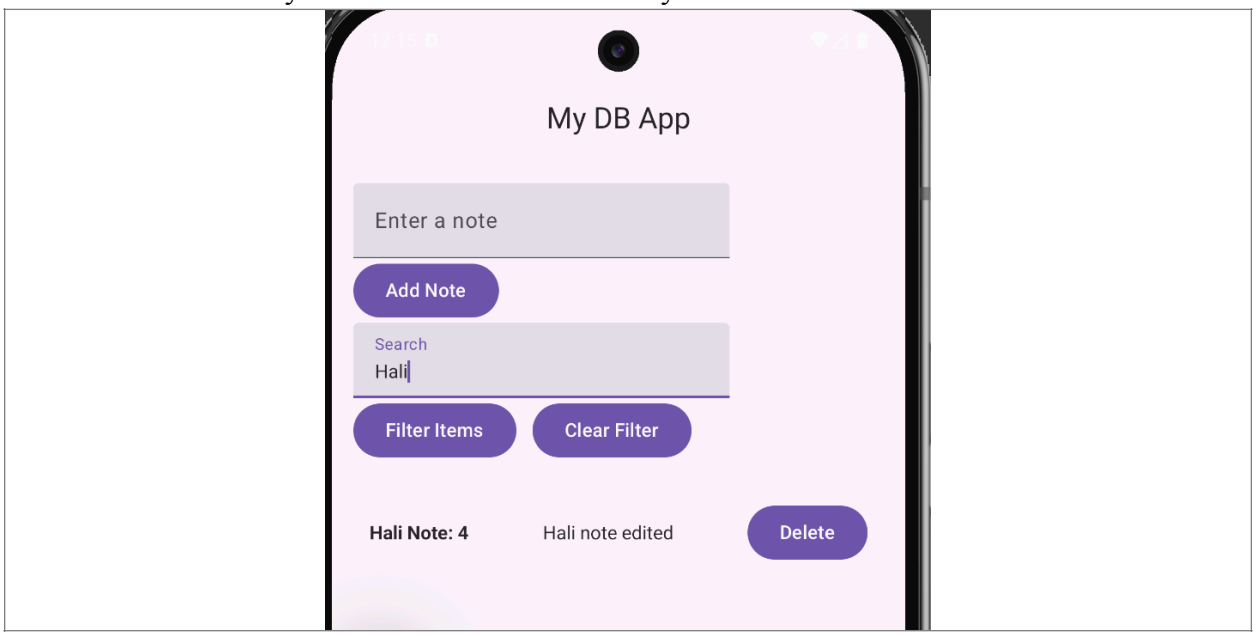
Figure out how to select multiple rows with this pattern:

```
"SELECT * FROM Note WHERE text LIKE '%' || :pattern || '%'"
```

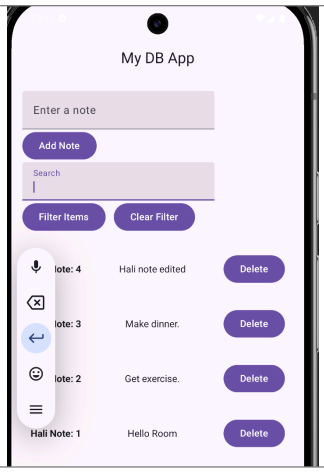
Make the changes so the output looks similar to the following:




Show a screenshot of your screen while filtered on your name:




Show a screenshot of your screen when the filter has been cleared:




Show your new UI code here:

```
Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
    TextField(
        value = searchPattern,
        onValueChange = { searchPattern = it },
        label = { Text("Search") }
    )
}
Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
    Button(onClick = {
        scope.launch(Dispatchers.IO) { reload() }
    }) { Text("Filter Items") }
    Button(onClick = {
        searchPattern = ""
        scope.launch(Dispatchers.IO) { reload() }
    }) { Text("Clear Filter") }
}
```

Show the new operator code that is added to your DAO:

```
@Query("SELECT * FROM Note WHERE text LIKE '%' || :pattern
|| '%'")

suspend fun searchNotes(pattern: String): List<Note>
```